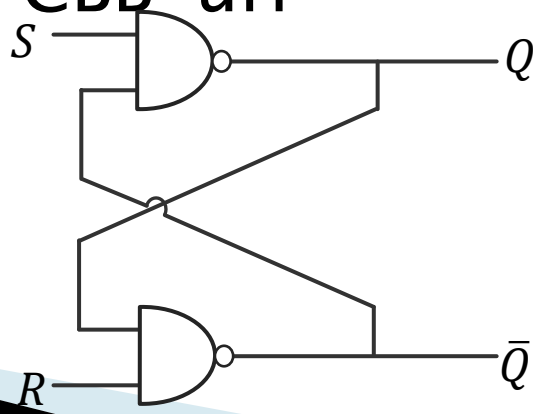
Proiectare Logică

I CB: Mariana Mocanu mariana.mocanu@upb.ro
I CC: Anca Morar anca.morar@upb.ro
I CA & CD: Costin Chiru costin.chiru@upb.ro

Curs 12: CBB J-K

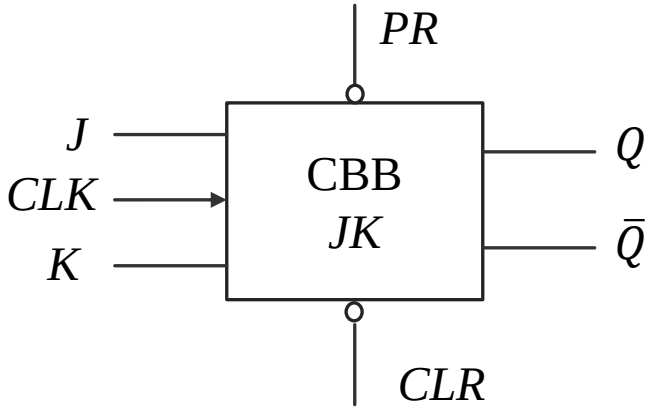
Reminder

- Automate secvențiale
  - Folosirea organigramelor în sinteza circuitelor secvențiale – 10 pași
- CBB-uri
  - D – ecuația de stare: $Q^{t+1} = D^t$
  - Variabilele de stare definesc starea
  - Numărul variabilelor de stare e legat de numărul stărilor
- Exemple cu CBB-uri
  - Interfonul



R	S	Q	!Q
0	0	1*	1*
0	1	0	1
1	0	1	0
1	1	Nu se modifică starea anterioară	

Circuitul basculant bistabil JK



- 2 Intrări de date: J, K
- 1 Intrare de ceas: CLK
- 2 Ieşiri: Q și !Q
- 2 Intrări de comandă: R (clear)
  S (preset)
  - asincrone, prioritare

- Implementarea fizică constă în 2 circuite pe pastilă
- Are funcționalitate asincronă precum celelalte CBB-uri
- Are un comportament derivat din funcționarea RS (unde aveam comportament neplăcut din cauza stării de incontrolabilitate)
- A apărut pentru a corecta problema de la RS (pentru ∀ combinație de intrare, se știe în ce stare trece CBB JK)

CBB JK – tabelul de adevăr

J^t	K^t	Q^t	Q^{t+1}	
0	0	0	0	Q^t
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	1	1
1	0	1	1	
1	1	0	1	$\overline{Q^t}$
1	1	1	0	

CLK	*J*	*K*	Q^{t+1}
↓	0	0	Q^t pasiv (NOP)
↓	0	1	**0 reset sincron**
↓	1	0	**1 set sincron**
↓	1	1	$\overline{Q^t}$ toggle

- Ne interesează funcționarea sincronă, determinată de J și K:
- Tabelul de adevăr este identic cu cel de la CBB RS, cu excepția J = K = 1, când ieșirea își modifică starea

CBB JK – ecuația de stare următoare

- Pe baza tabelului de adevăr încercăm să determinăm ecuația de stare următoare → completăm diagrama Karnaugh corespunzătoare

J^t	K^t	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q \ JK	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$Q^{t+1} = !KQ^t + J!Q^t$$

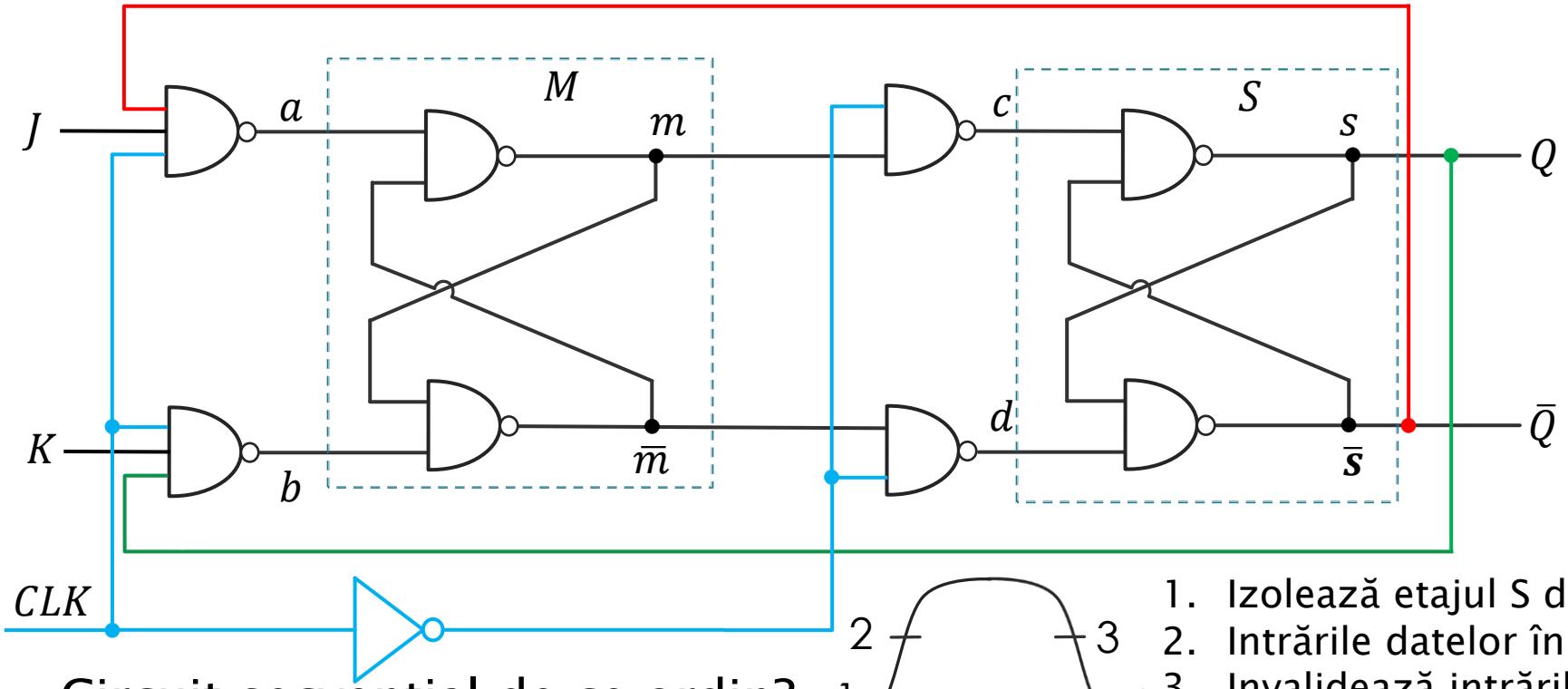
A 3-a ecuație pe care trebuie să o memorați

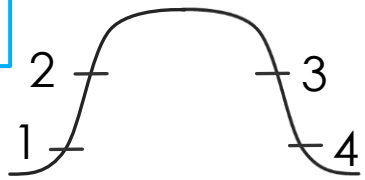
CBB – determinare J și K

- Cel mai răspândit tip de CBB
- Ca la RS, în general se cunoaște funcționarea circuitului (Q^t, Q^{t+1}) și se dorește identificarea elementelor de circuit necesare pentru a se implementa J^t și K^t.
- Acestea se determină fie pe baza:
  - Ecuației de stare următoare $Q^{t+1} = (J!Q + !KQ)^t$:
    - Dacă $Q^t = 0 \Rightarrow Q^{t+1} = J^t$ sau $J^t = Q^{t+1}$ pentru $Q^t = 0$
    - Dacă $Q^t = 1 \Rightarrow Q^{t+1} = !K^t$ sau $K^t = !Q^{t+1}$ pentru $Q^t = 1$
  - Tabelului:

Q^t	Q^{t+1}	J	K
0	0	0	–
0	1	1	–
1	0	–	1
1	1	–	0

CBB JK – implementare (1)





1. Izolează etajul S de M
2. Intrările datelor în M
3. Invalidează intrările
4. Trecerea datelor M → S

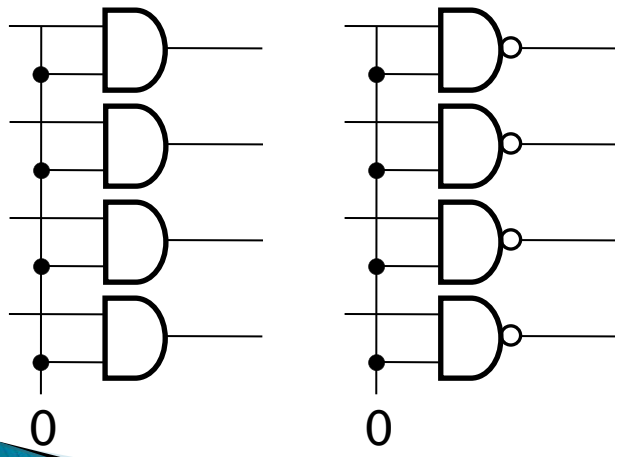
- Circuit secvențial de ce ordin?
- O luăm pe bucățele:
  - Zonele punctate reprezintă CBB-uri elementare (de ordin 1)
  - Cum sunt legate? În serie → ordin 1
  - DAR! Observăm cele 2 conexiuni de reacție (!Q – J & Q – K)
  - → Ordinul este 2 pentru că are în bucla de reacție CBB de ordin 1

CBB JK – implementare (2)

- Avem 2 module: master (M) și slave (S)
- Am vorbit în trecut de sincronizarea
  - Pe front
  - Master-slave (în 4 timpi). Care sunt acțiunile din cei 4 timpi?
    - 1. decuplarea M de S,
    - 2. intrarea datelor în M,
    - 3. invalidarea intrărilor,
    - 4. trecerea datelor din M în S
    - Această sincronizare are ca scop protejarea circuitului de momente ce țin de timpii de propagare, timpii de stabilizare, momentele când tensiunea este între nivelele de 0 și 1, etc.

CBB JK – sincronizare (1)

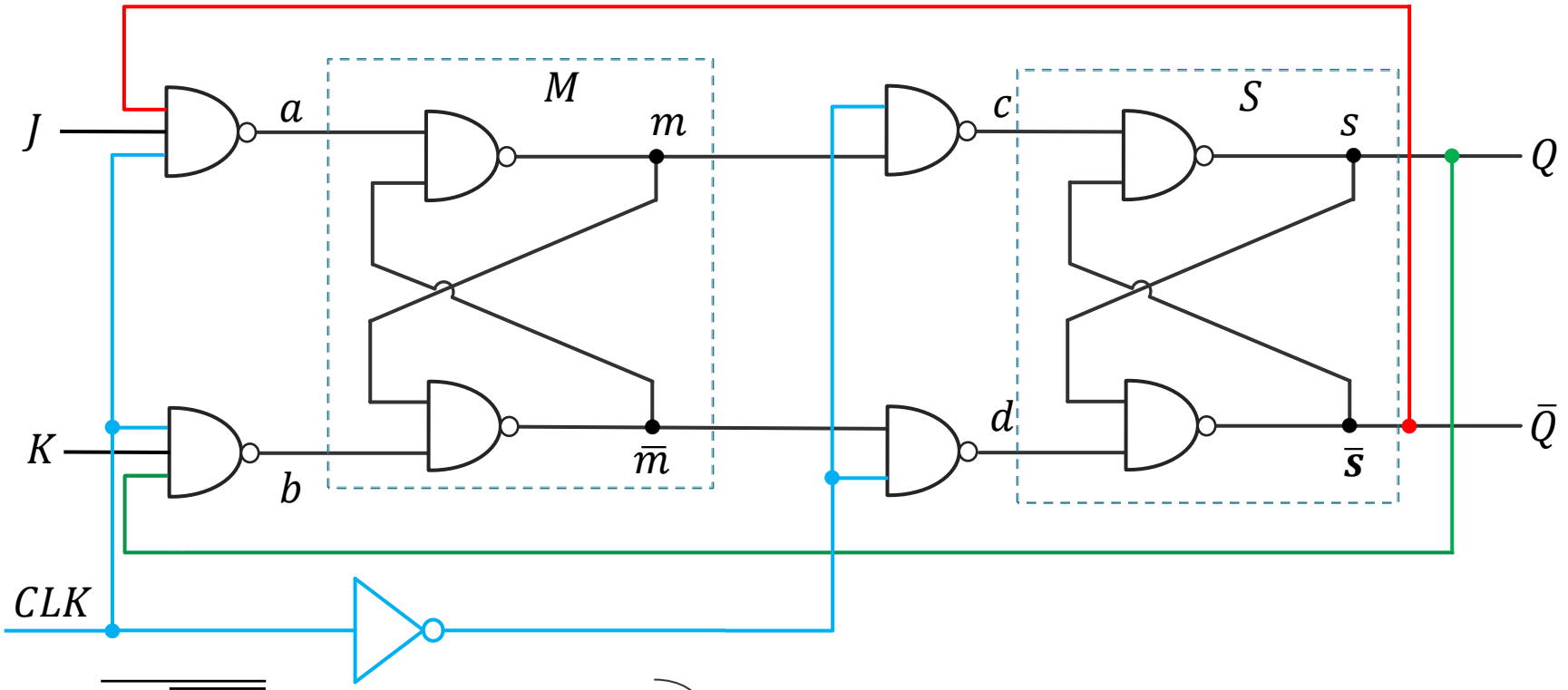
- Ce înseamnă invalidarea intrărilor? Cum se face?
- Porțile ȘI pot bloca aplicarea datelor utile prin simpla aplicare a unei valori 0. La NAND e la fel, ieșirea nefiind 0, ci 1.



- Este foarte folosită pentru că protejează circuitul și este foarte simplă (ieftină, fără erori)
- Așa se întâmplă cu nivelul a-b / c-d de NAND-uri din CBB JK

CBB JK – sincronizare (2)

- Care sunt datele utile?
- J & K
- Cum blocăm circuitul?
- Cu ajutorul CLK
- Cum adică?
- O dată e valid un nivel, o dată celălalt
- Cum se face asta?
- Cu ajutorul inversorului → el asigură decuplarea



$$s = \overline{c * \overline{s * d}} = \bar{c} + s * d$$

$$m = \overline{a * \overline{b * m}} = \bar{a} + b * m$$

$$c = \overline{m * \overline{CLK}}$$

$$d = \overline{\bar{m} * \overline{CLK}} = m + \mathrm{CLK}$$

$$a = \overline{J * CLK * \bar{s}} = \overline{J * CLK * \bar{s}}$$

$$b = \overline{K * CLK * s} = \overline{K} + \overline{CLK} + \bar{s}$$

Determinăm formulele pentru s și m:

$$s = m * \overline{CLK} + m * s + CLK * s$$

$$m = J * CLK * \bar{s} + \overline{K} * m + \overline{CLK} * m + \bar{s} * m$$

CBB JK – funcționare

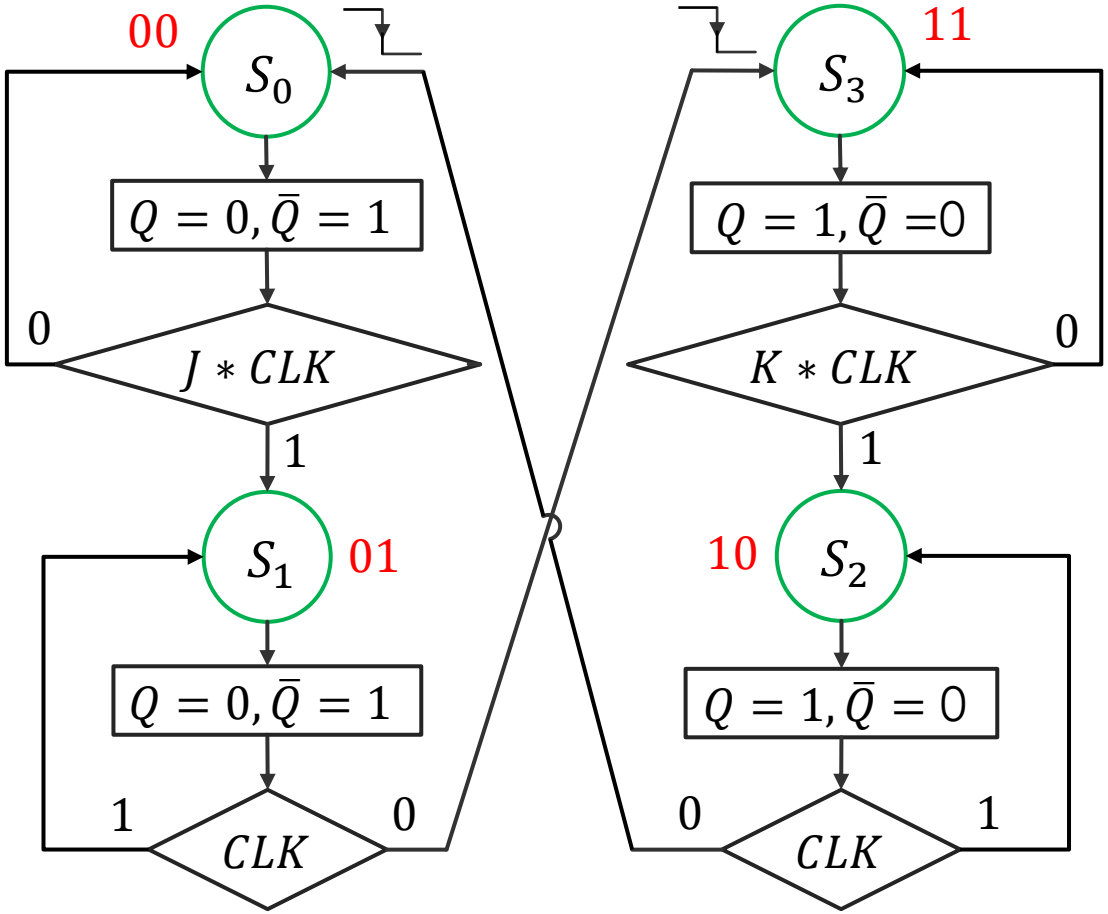
- Putem considera m și s ca fiind variabilele de stare ale CBB-urilor elementare → circuitul are 4 stări corespunzătoare acestor variabile de stare.
- Construim diagrama Karnaugh folosind s, m, CLK, J și K

$$s = m * \overline{CLK} + m * s + CLK * s$$

$$m = J * CLK * \bar{s} + \overline{K} * m + \overline{CLK} * m + \bar{s} * m$$

s, m \ Clk, J, K	0 0 0		0 0 1		0 1 1		0 1 0		1 1 0		1 1 1		1 0 1		1 0 0		
00	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	S_0
01	1	1	1	1	1	1	1	1	0	1	0	1	0	1	0	1	S_1
11	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	S_3
10	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	S_2

CBB JK – Organigrama



- Cele 4 momente de la sincronizarea master-slave corespund celor 4 stări.
- Temă: care moment corespunde cărei stări? + Justificare.

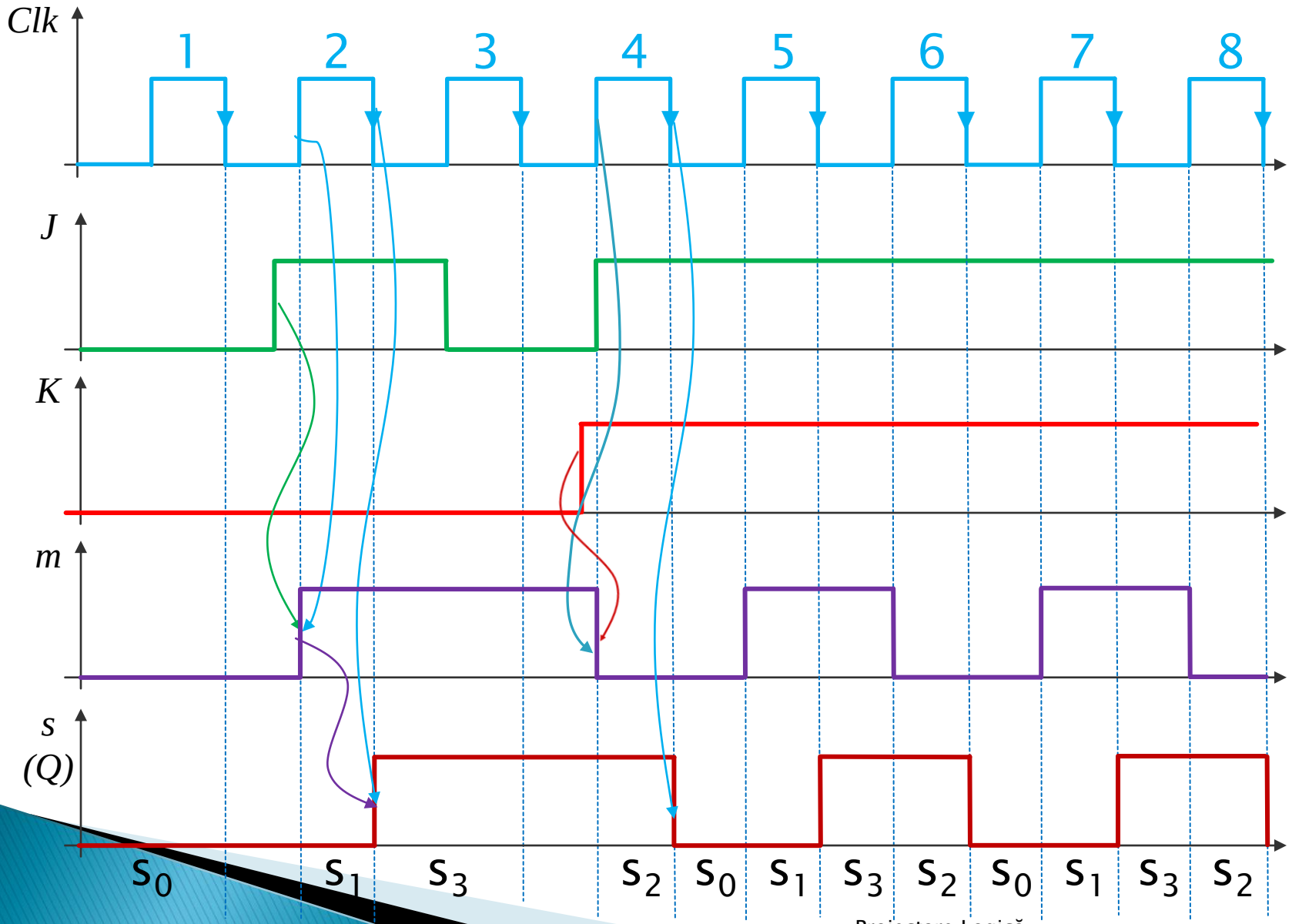
$$s = m * \overline{CLK} + m * s + CLK * s$$

$$m = J * CLK * \bar{s} + \overline{K} * m + \overline{CLK} * m + \bar{s} * m$$

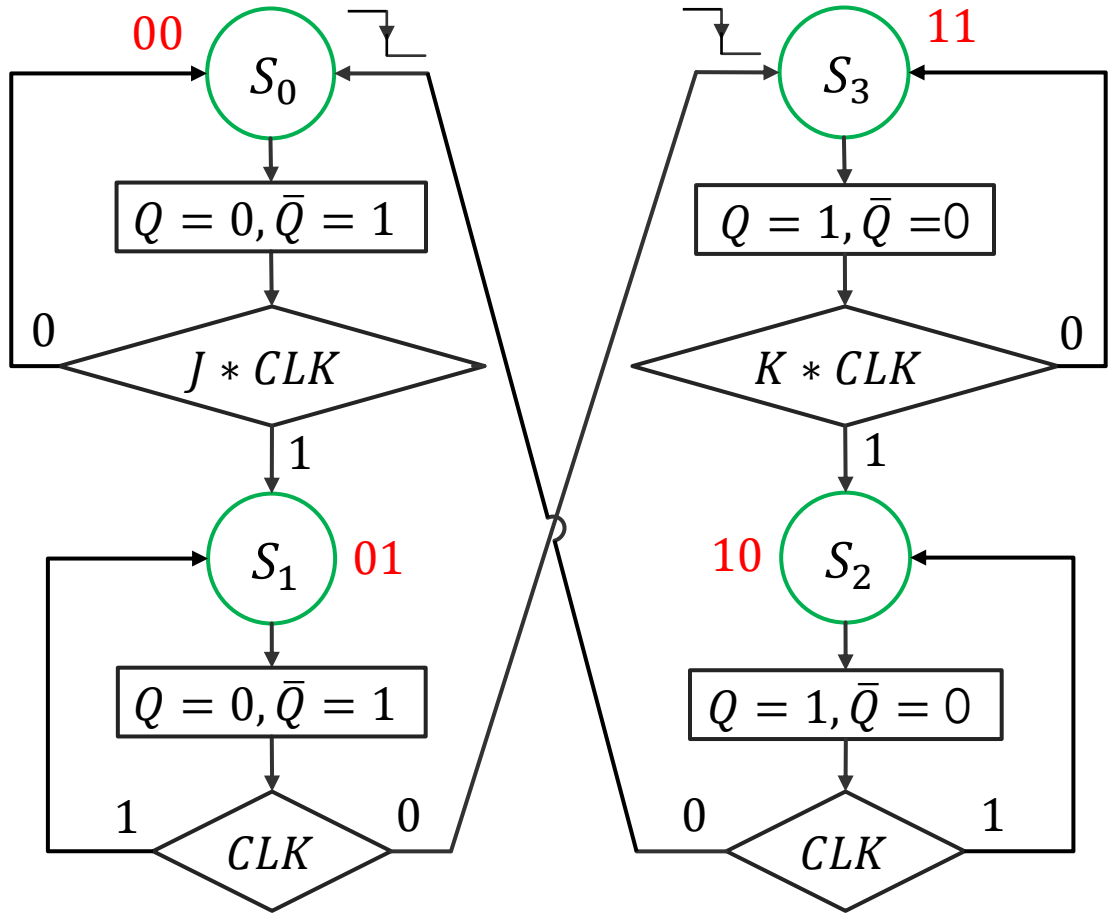
CBB JK – funcționare

- Tranziția mărimilor de ieșire se manifestă pe căderea impulsului de ceas → funcționarea circuitului este sincronă pe frontul descrescător al semnalului CLK
- Când CLK = 0, circuitul este fie în S_0, fie în S_3, când CLK = 1, circuitul este fie în S_1, fie în S_2
- J controlează S_0 a circuitului (cât timp J = 0, circuitul rămâne blocat în S_0) → J controlează trecerea lui Q din 0 în 1
- K controlează S_3 a circuitului (cât timp K = 0, circuitul rămâne blocat în S_3) → K controlează revenirea lui Q din 1 în 0
- Obs: J și K au funcționalitate similară cu S și R, dar nu pot determina funcționarea circuitului în lipsa ceasului (acțiunea lor e de tip sincron)

CBB JK – Diagrama de semnal



CBB JK – Organigrama

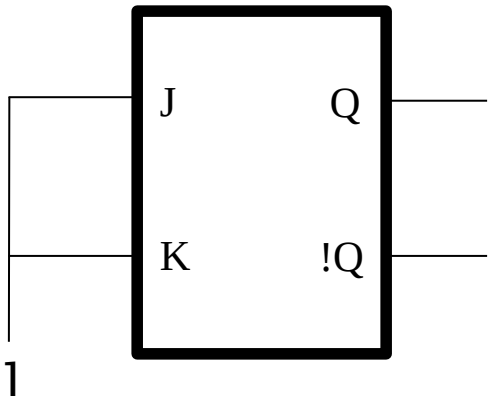


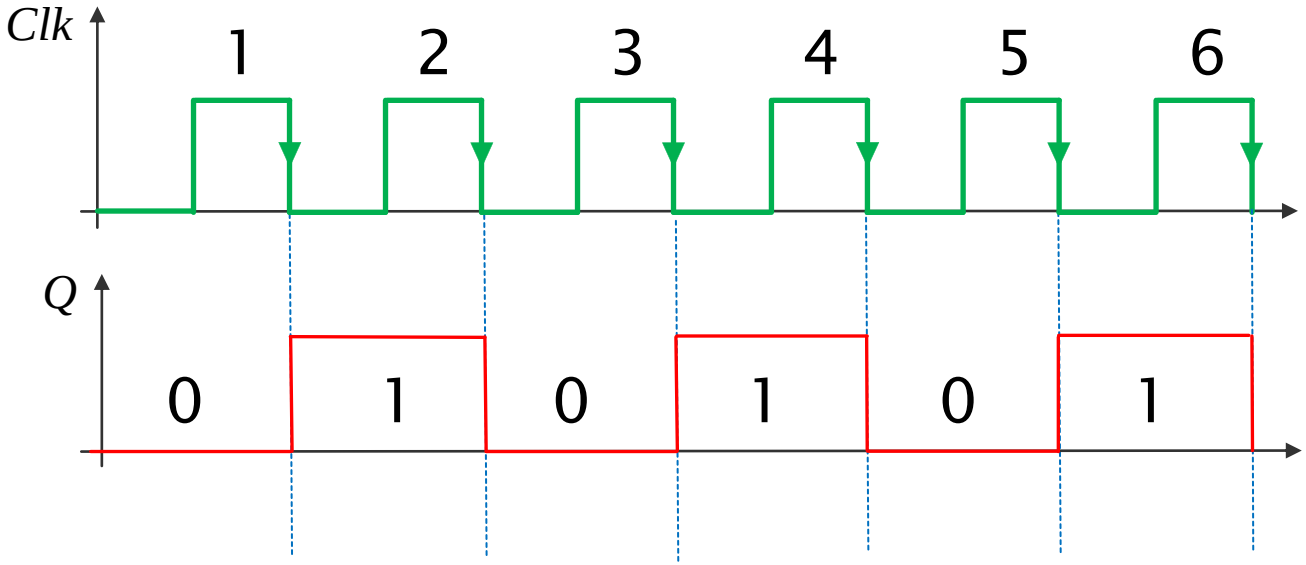
CBB JK – explicații

- Circuitul M captează informația utilă pe durata activă a lui CLK (= 1)
- Circuitul S copiază informația din M în momentul trecerii ceasului în 0
- În practică, circuitul este prevăzut și cu comenzi asincrone (preset și clear) active pe 0:

PR	*R*	*J*	*K*	*CLK*	*Q t+1*	
1	1	*	*	↓	Ca mai devreme	← Funcționare sincronă
0	1	*	*	*	1 (set asincron)	Funcționare asincronă
1	0	*	*	*	0 (reset asincron)	Funcționare asincronă
0	0					← Nepermis

CBB JK – aplicație (1)



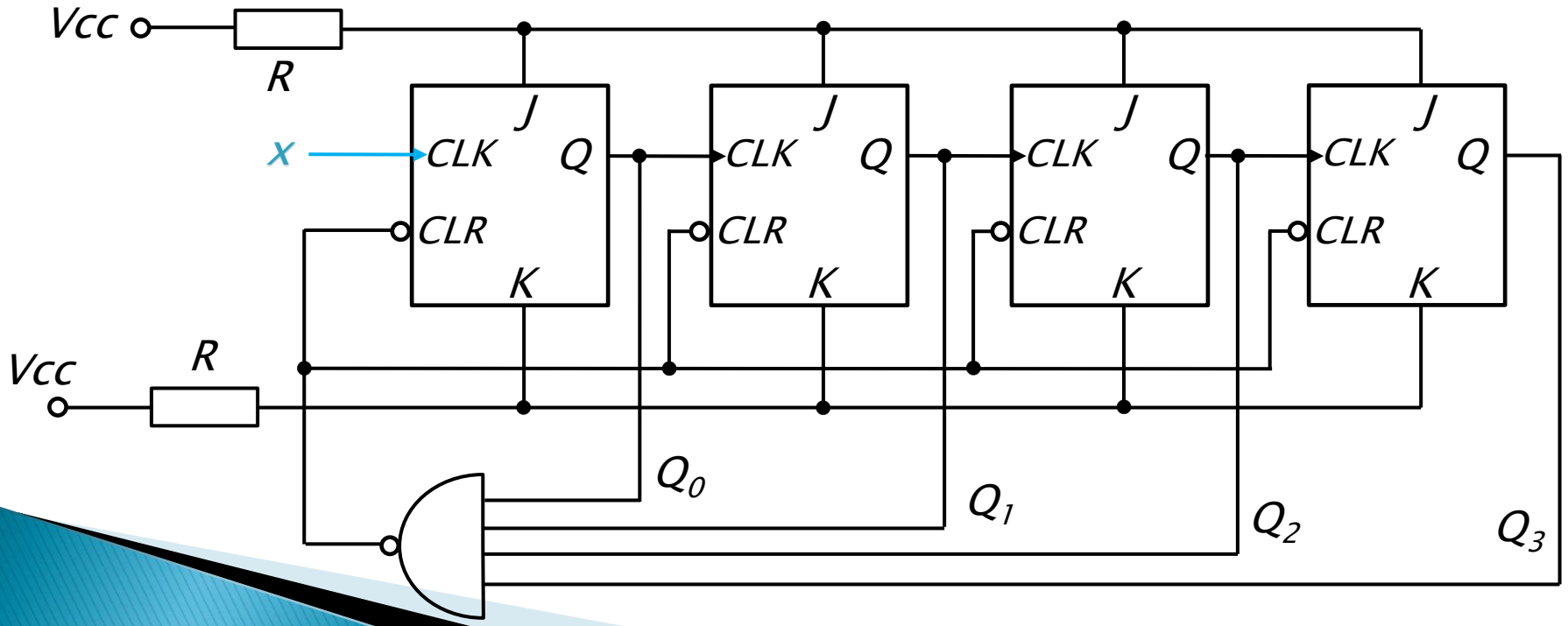


- Ce face?
- Inversează valoarea lui Q: $Q^{t+1} = !Q^t$
- Divizor de frecvență
- Am mai intâlnit?
- Da, la CBB D
- Există vreo diferență intre ele?
- Da. Frontul tranziției

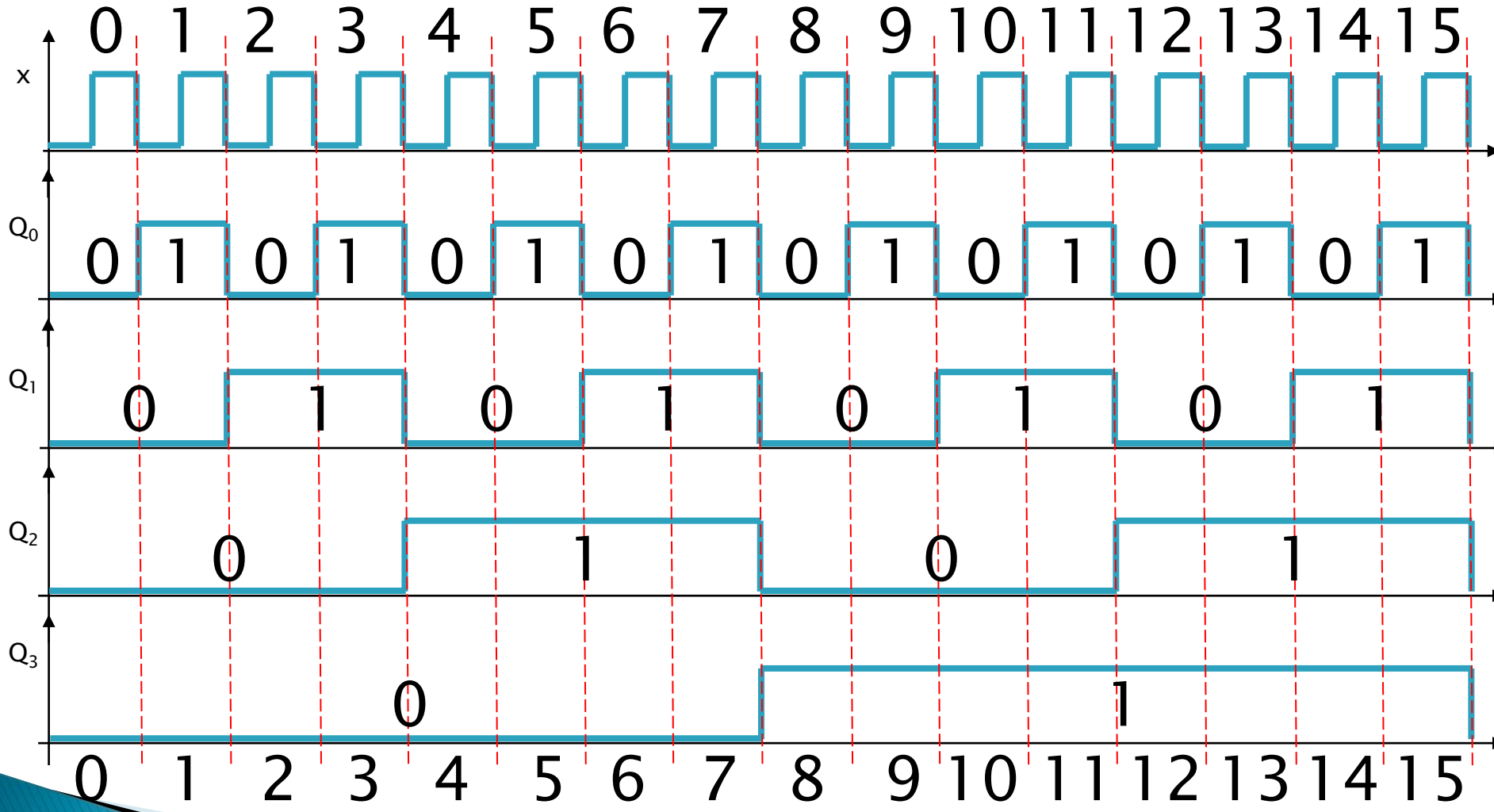
$Q^{t+1} = !KQ^t + J!Q^t$

CBB JK – aplicație (2) – Numărător pe 4 biți

- Schemă clasică de numărător
- Când face reset? Când toate Q-urile sunt 1
- Numărător ciclic → după ce ajunge la 1111 trece în 0



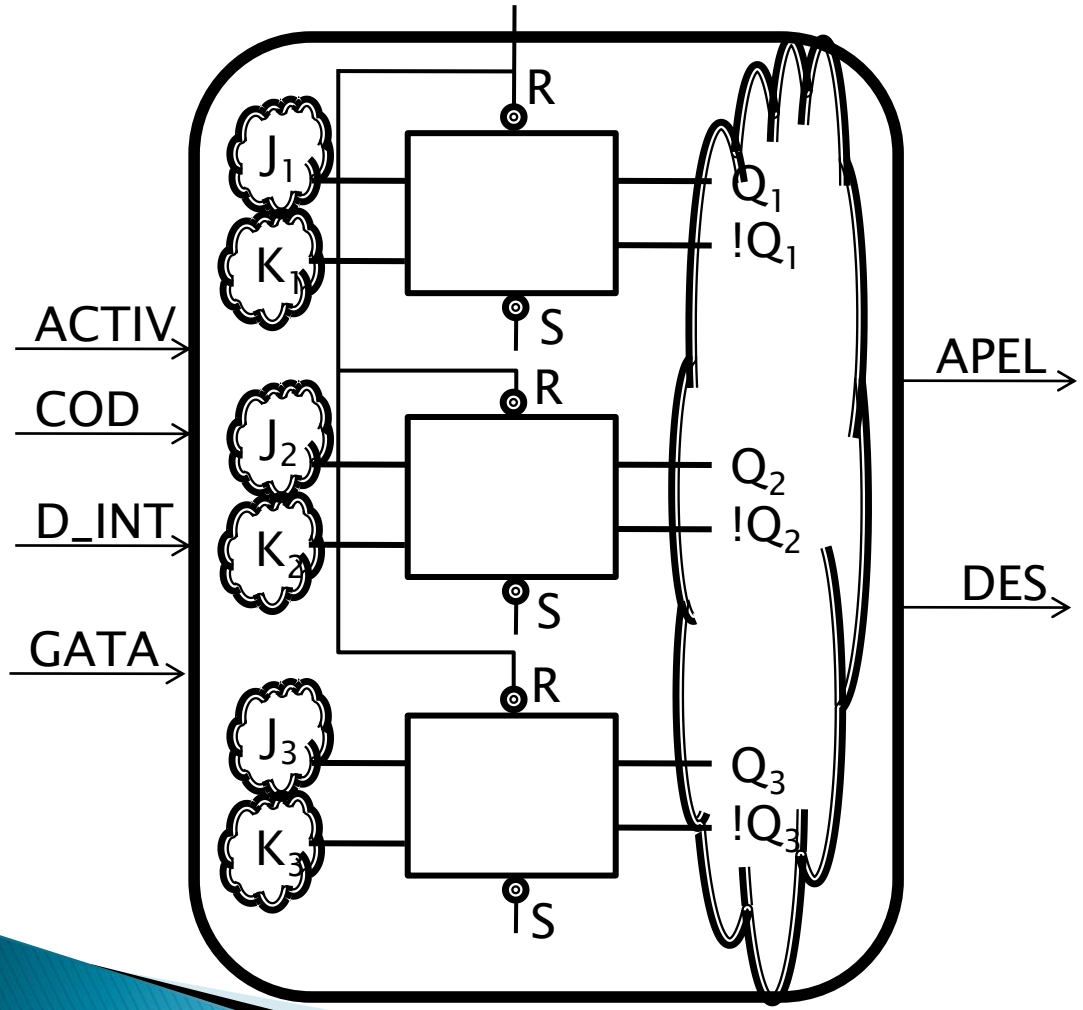
Numărător – Diagrama de semnal



Numărător – detalii

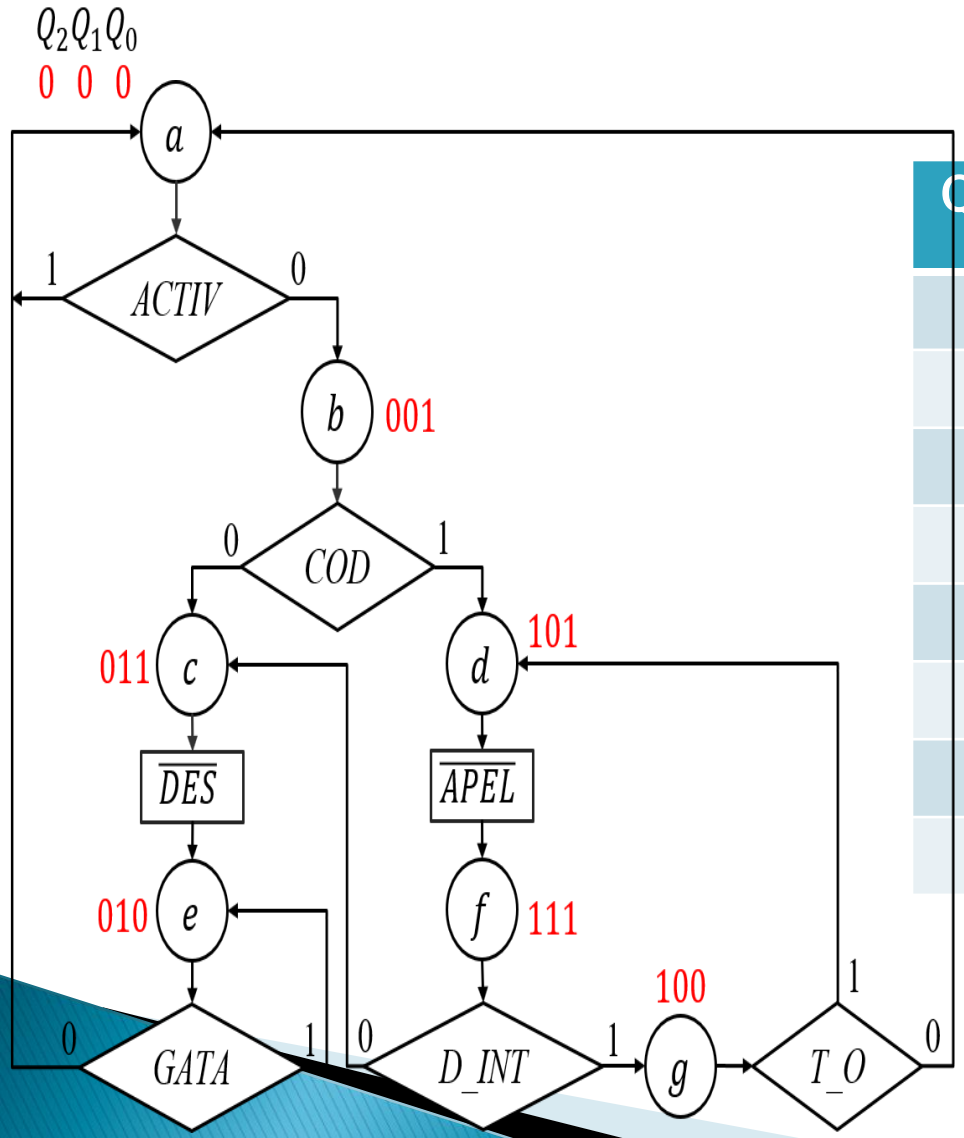
- Care este cel mai semnificativ bit și care cel mai puțin semnificativ bit?
- MSB: Q_3, LSB: Q_0
- În funcție de ce punem în poarta de jos se modifică valoarea maximă până la cât numără
- Ex: ca să fac numărător decimal, trebuie să neg intrările de la Q_1 și Q_2
- Putem să îl extindem la mai mulți biți?
- Dezavantaj: fiecare circuit JK introduce întârzieri și dacă semnalul x este prea rapid se pierd din semnale

CBB JK – aplicație (3) – Interfon



- Similar cu CBB de tip D, trebuie să detectăm ce punem pe intrări (J,K) și cum combinăm ieșirile (Q,!Q)
- La CBB D era foarte simplu de determinat intrările ($Q^{t+1} = D^t$)
- La CBB JK trebuie să împărțim Q^{t+1} în intrarea lui J și a lui K

Interfon – reminder



Q_2^t	Q_1^t	Q_0^t	Q_2^{t+1}	Q_1^{t+1}	Q_0^{t+1}
0	0	0	0	0	!ACTIV
0	0	1	COD	!COD	1
0	1	0	0	GATA	0
0	1	1	0	1	0
1	0	0	T_O	0	T_O
1	0	1	1	1	1
1	1	0	*	*	*
1	1	1	D_INT	! D_INT	! D_INT

Interfonul (3)

- Cum împărțim?

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

$Q_2 = Q_2!Q_1Q_0 + T_O*Q_2!Q_1 + COD*!Q_1Q0 + D_INT*Q_2Q_0$

- Știm că $Q^{t+1} = J!Q^t + !KQ^t$
- Ce se întâmplă cu Q^{t+1} dacă $Q^t = 0$? $Q^{t+1} = J$
- Ce se întâmplă cu Q^{t+1} dacă $Q^t = 1$? $Q^{t+1} = !K$
- Citim invers: $J = Q^{t+1}|_{Q^t = 0}$ și $!K = Q^{t+1}|_{Q^t = 1}$
- $J = Q^{t+1}$ dar numai partea corespunzătoare lui $Q^t = 0$ → unde e $Q^t = 0$? Pentru Q_2 = stânga

Interfonul (4)

- Ecuația de stare exprimă superpoziția a 2 componente (când $Q^t = 0$ și $Q^t = 1$).
- Am mai întâlnit asta? Da! Superpoziția de la diagrame Karnaugh cu variabile externe
- Orice diagramă Karnaugh poate fi împărțită în două zone care nu se intersectează ci se complementează și acoperă întreaga diagramă, în funcție de $Q^t = 0$ și $Q^t = 1$
- Din analiza ecuației de stare următoare rezultă că J și K se identifică distinct cu una dintre zone
- → *ca să știm ce aplicăm pe intrările CBB JK lucrăm cu diagrama de stare următoare pe care identificăm zona pentru J și zona pentru !K*

Interfonul (5)

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	T_O
1	COD	0	D_INT	1

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	*
1	COD	0	*	*

$J_2 = COD!Q_1Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	T_O
1	*	*	D_INT	1

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	!T_O
1	*	*	!D_INT	0

$K_2 = !T_O\ !Q_0 + !D_INT\ Q_1$

- Pornim de la diagrama de stare
- Care e zona pentru J2?
- Primele 2 coloane → restul nu ne interesează → punem *
- La fel pentru !K
- Putem face direct diagrama lui K prin negarea fiecărei căsuțe în parte → de obicei așa se face ca să nu greșiți!!!

Interfonul (6)

- Cum se face împărțirea pentru J_1 și K_1?
- E la fel ca pentru Q_2?

Q_0 \ Q_2Q_1	00	01	11	10
0	0	Gata	*	0
1	!COD	1	!D_INT	1

$$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	*	*	0
1	!COD	*	*	1

$$J_1 = Q_2Q_0 + !COD*Q0$$

Q_0 \ Q_2Q_1	00	01	11	10
0	*	!Gata	*	*
1	*	0	D_INT	*

$$K_1 = !Gata*!Q_0 + D_INT*Q_2$$

Interfonul (7)

- Cum se face împărțirea pentru J_0 și K_0?

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	1	0	!D_INT	1

$$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$$

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	*	*	*	*

$$J_0 = !ACTIV*!Q_2!Q_1 + T_O*Q_2$$

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	*
1	0	1	D_INT	0

$$K_0 = !Q_2Q_1 + D_INT*Q_1$$

Interfonul (8)

- OBSERVAȚIE: prin spargerea lui Q^{t+1} în zonele corespunzătoare lui J și K avem niște formule care ne duc la o complexitate mai mică → circuitele pe intrările lui J și K sunt mai simple decât ce puneam pe intrarea lui D, dar trebuie să fim atenți cum facem împărțirea (și să nu uităm să negăm)

Implementarea Q^{t+1} circuitului cu porți

$J_2 = COD!Q_1Q_0$

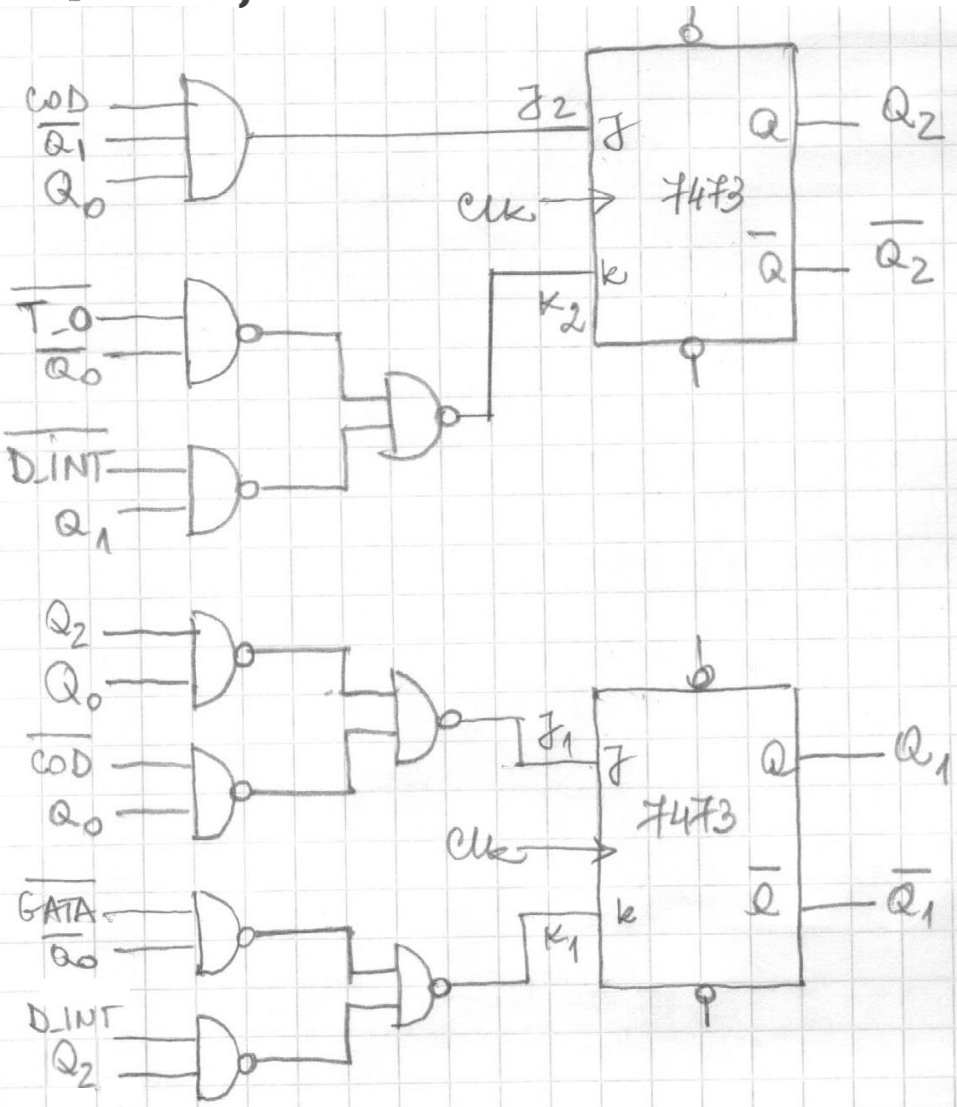
$K_2 = !T_O\ !Q_0 + !D_INT\ Q_1$

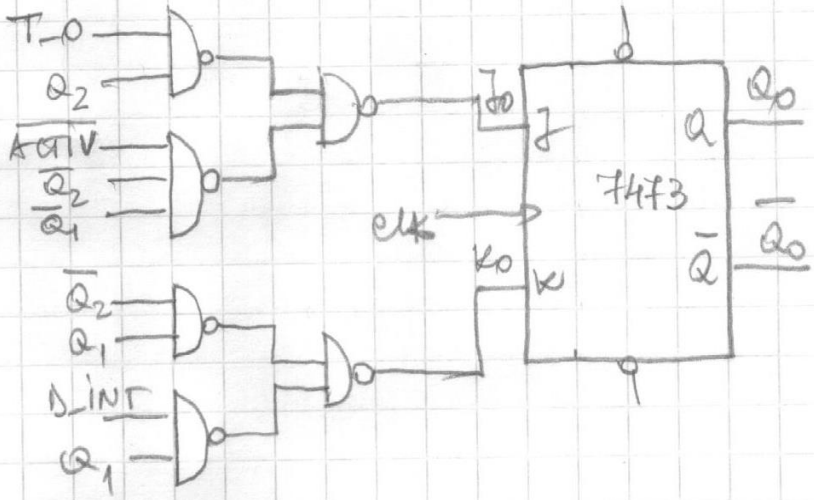
$J_1 = Q_2Q_0 + !COD*Q0$

$K_1 = !Gata*!Q_0 + D_INT*Q_2$

$J_0 = !ACTIV*!Q_2!Q_1 + T_O*Q_2$

$K_0 = !Q_2Q_1 + D_INT*Q_1$

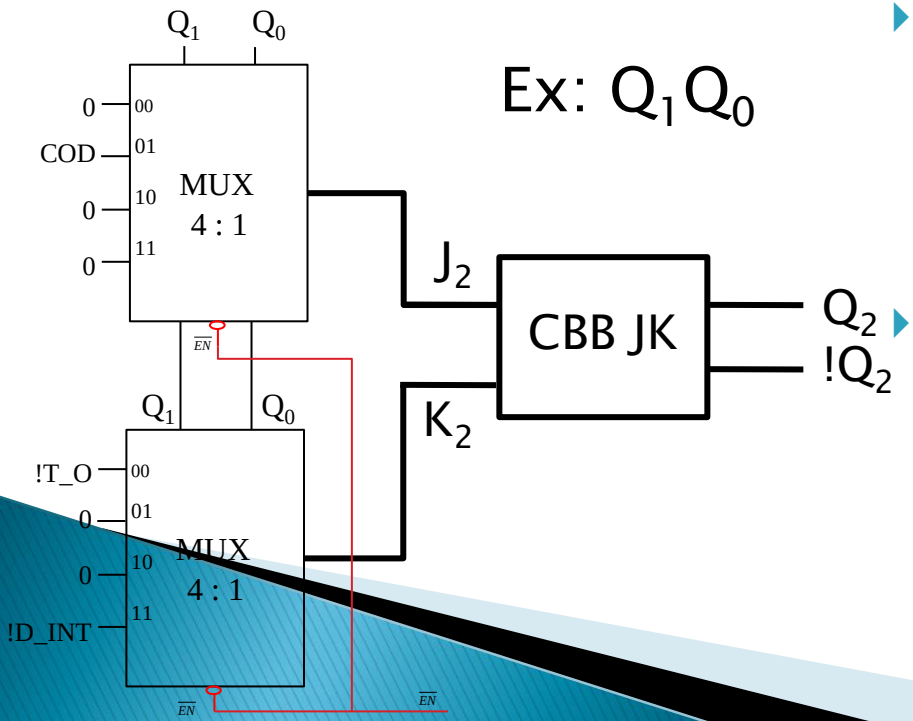




Implementarea cu CBB JK și MUX

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	*
1	COD	0	*	*

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	!T_O
1	*	*	!D_INT	0



- În practică, pentru cele 2 MUX se aleg aceleași variabile de selecție dar la examen le putem cere diferite (în scop didactic)
- Atenție: Pe condițiile de la MUX nu se pun variabile externe (ex: Gata, Cod etc.), ci doar variabile de stare
- Mereu ne ocupăm de partea secvențială și apoi de cea combinațională. Începem cu CBB și apoi MUX → reverse engineering
- Verificare: nu pot să apară variabilele de selecție pe intrări → termenul minimizat trebuie să conțină termenul de pe intrare + variabilele de selecție în combinația dată

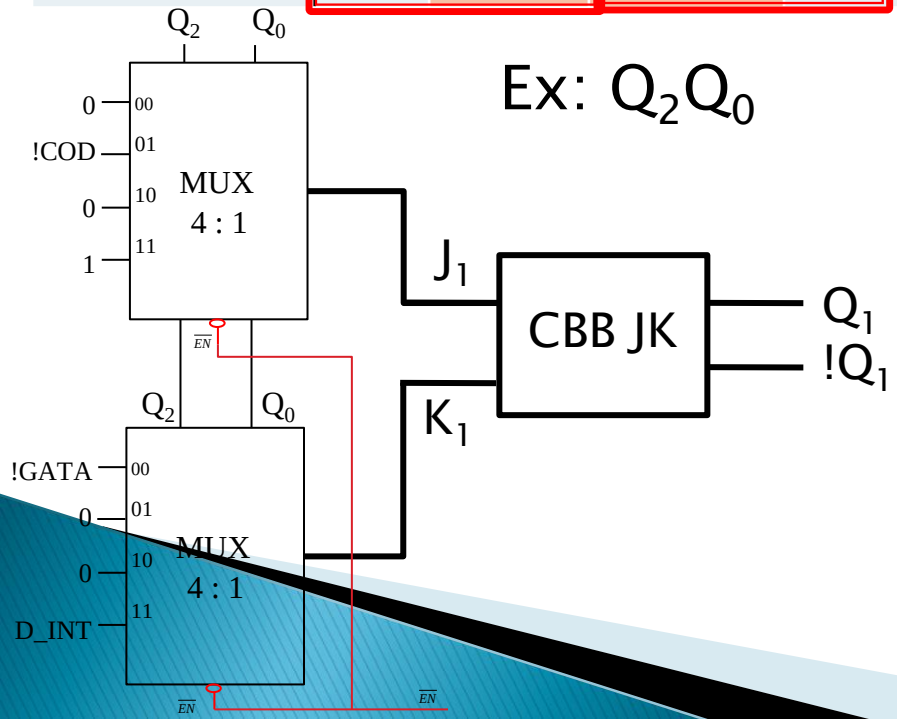
Implementarea cu CBB JK și MUX

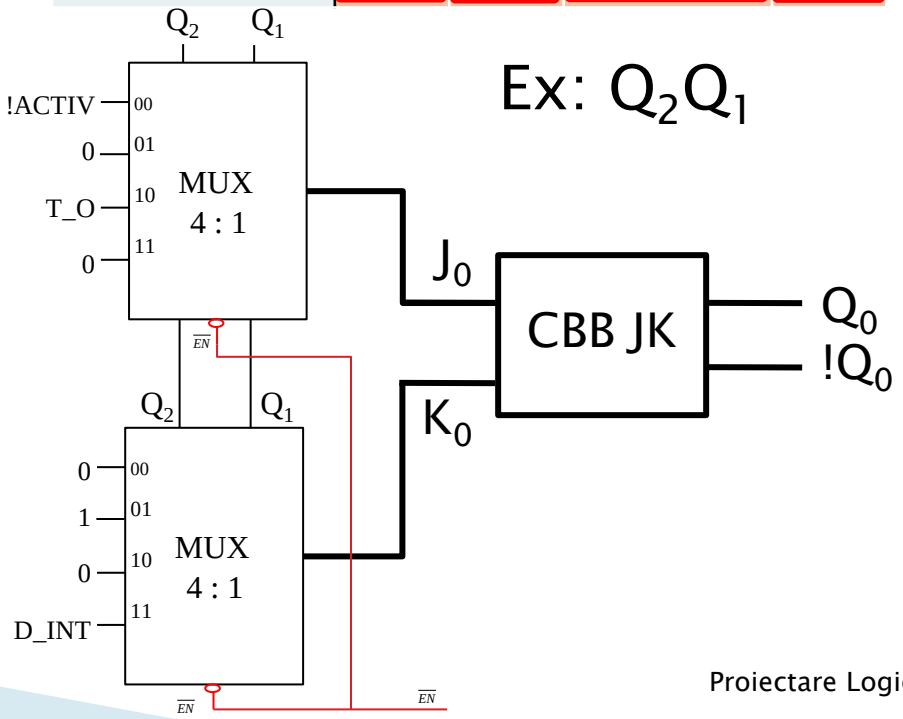
Q_0 \ Q_2Q_1	00	01	11	10
0	0	*	*	0
1	!COD	*	*	1

Q_0 \ Q_2Q_1	00	01	11	10
0	!ACTIV	0	*	T_O
1	*	*	*	*

Q_0 \ Q_2Q_1	00	01	11	10
0	*	!Gata	*	*
1	*	0	D_INT	*

Q_0 \ Q_2Q_1	00	01	11	10
0	*	*	*	*
1	0	1	D_INT	0





Interconectare MUX – CBB

- La examen, în enunțul problemelor pot varia doar tipul de CBB și modul de realizare a circuitelor combinaționale.
- Ex: implementați Q_i folosind
  - CBB D și porți (eventual de un anume fel);
  - CBB D și MUX (de un anume tip);
  - JK și porți / MUX
  - JK și porți pentru J, respectiv MUX pentru K
- În cei 10 pași am delimitat ecuațiile de stare următoare de implementarea cu CBB-uri (la tip D era același lucru, dar la JK / RS trebuie să împărțim în 2)

Interfonul (9) – ieșirile

- Ieșirile circuitului rămân la fel ca în cazul CBB D (ca să fie active pe 0, trebuie negate după minimizare sau trebuie construită diagrama negată):

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	1	0	0

$DES = !Q_2Q_1Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	0	1

$APEL = Q_2!Q_1Q_0$

- Alternativa este să folosim decodificatorul 7442 pentru a implementa ambele ieșiri

ÎNTREBĂRI?